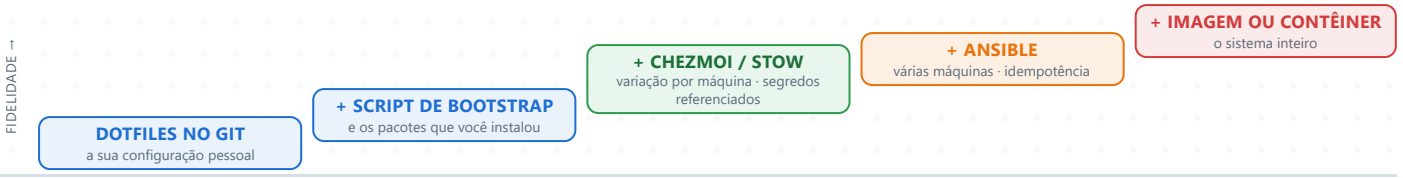
DOTFILES E AMBIENTES REPRODUZÍVEIS

A VM tinha três anos: quarenta aliases refinados um a um, um `.vimrc` lapidado em dezenas de sessões, scripts em `~/bin` que ninguém mais entendia e funcionavam. O disco do hospedeiro falhou numa terça; o snapshot era de quatro meses antes. ***Reinstalar o Kali leva vinte minutos — três anos de decisões invisíveis, não.***

097
GUIAS LINUX

O ESPECTRO DE REPRODUTIBILIDADE: CADA DEGRAU REPRODUZ MAIS E CUSTA MAIS



O erro comum é pular direto para o topo. Uma imagem de contêiner reproduz o ambiente perfeitamente e é *péssima* para configuração que você ajusta todo dia. **Para uma estação de trabalho pessoal, o equilíbrio fica entre o segundo e o terceiro degrau** — dotfiles versionados mais um script que instala os pacotes; Ansible e imagens entram quando as máquinas passam a ser várias. **E o segundo degrau é barato:** `apt-mark showmanual` lista só o que *você* decidiu instalar, ignorando as dependências — a diferença entre uma lista útil de 120 linhas e um despejo de 2.000 pacotes.

O QUE ENTRA NO REPOSITÓRIO — E POR QUAL ESTRATÉGIA

CONFIGURAÇÃO ESCRITA POR VOCÊ `.zshrc` · `.vimrc` · `.tmux.conf` · `.gitconfig` · `~/bin`	**SIM**
ESTADO GERADO PELO PROGRAMA `.zsh_history` · `.cache/` · `.local/state/`	**NÃO**
SEGREDO `.ssh/id_*` · `.gnupg/` · `.netrc` · tokens	**NUNCA**
DADOS DE APLICAÇÃO `.mozilla/` · `.msf4/` · `.local/share/`	**NÃO**

Segredo não entra nem em repositório privado — privado vira público por engano com frequência incômoda, e commit é permanente mesmo depois de removido. A ponte de melhor custo-benefício resolve 90% dos casos: `[[ -r ~/.zshrc.local ]] && source ~/.zshrc.local` — o repositório guarda a **estrutura**, a máquina guarda o **segredo**. Para o resto, `pass` e `age`.

ESTRATÉGIA	MELHOR PARA · O QUE CUSTA
links + script	começar, com controle total · você escreve o script (ou usa o **GNU Stow**, que dispensa escrevê-lo em troca de uma estrutura rígida). Editar o arquivo no home **é** editar o repositório
repositório *bare*	não gostar de links: o `.git` fica à parte, a árvore é o home · `git` sem o alias, no home, confunde
chezmoi / yadm	várias máquinas e segredos referenciados, com modelos por host · aprender a ferramenta

Máquina-específico sem duplicar arquivo: ramifique por `ID` do `/etc/os-release` (`apt` × `dnf`, caminhos, serviços) e **teste se a ferramenta existe antes de criar o alias**: `command -v bat >/dev/null && alias cat='bat'`. Um `.zshrc` que assume ferramenta presente cospe erro em cada terminal da máquina que não a tem.

A regra é lista de *inclusão*, nunca lista de exclusão. Dar `git init` no home e ir acrescentando ao `.gitignore` conforme os problemas aparecem é o caminho garantido para vazar `.ssh/id_ed25519` ou um `.zsh_history` cheio de comandos com senha: o `.gitignore` protege apenas do que você *lembrou* de listar, e você não vai lembrar de tudo. Faça o inverso — `.gitignore` com `*` na primeira linha e cada arquivo entrando com `git add -f`. Assim o padrão é "não versionado" e a exceção é deliberada, o que inverte quem paga o preço do esquecimento.

NA PRÁTICA — CONSTRUIR, E DEPOIS *PROVAR* QUE FUNCIONA

kali@kali: ~/.dotfiles

```
$ git init
$ printf '*\n!*/\n!.gitignore\n' > .gitignore
# padrão: NADA entra. Cada arquivo, deliberado:
$ git add -f zsh/.zshrc vim/.vimrc git/.gitconfig
$ git ls-files  # confira o que está rastreado
# o que os dotfiles NÃO capturam: os pacotes
$ apt-mark showmanual > pacotes-manuais.txt
$ xargs -a pacotes-manuais.txt sudo apt install -y
# o instalador educado guarda o que já existia:
[[ -e $destino && ! -L $destino ]] \
  && mv -- "$destino" "$destino.pre-dotfiles"
ln -sfn -- "$origem" "$destino"
$ git log -p --all | grep -iE 'BEGIN .*PRIVATE|token='
# a última checagem ANTES de qualquer git push
```

Um repositório de dotfiles que nunca foi aplicado numa máquina limpa é uma *hipótese*, não um backup. O teste real leva dois minutos: um contêiner descartável (`docker run --rm -it kalilinux/kali-rolling bash`) ou um usuário novo na própria máquina — `useradd -m teste-dot`, aplicar o instalador como ele, `userdel -r` no fim. **O que esse teste sempre revela na primeira vez:** uma dependência que você tinha e esqueceu de listar, um caminho absoluto com o *seu* nome de usuário dentro, e uma linha que assume uma ferramenta presente. É muito melhor descobrir isso num contêiner de teste do que na máquina nova, no dia em que a antiga morreu.

Um bootstrap completo tem quatro partes, nesta ordem: 1) pacotes (`xargs -a pacotes-manuais.txt sudo apt install -y`); **2)** dotfiles (`git clone` + `instalar.sh`); **3)** ajustes do sistema que não são dotfile (fuso, teclado, serviços); **4) verificação** — `command -v vim tmux git zsh` e falha explícita se faltar algo. Sem ela você não sabe se deu certo — só que o script terminou.

RESUMO RÁPIDO

- ✓ **O ambiente de trabalho costuma ser a única parte da infraestrutura que não é reproduzível** — e três anos de ajustes invisíveis somem com um disco.
- ✓ **Lista de inclusão, nunca de exclusão:** `*` na primeira linha do `.gitignore` e cada arquivo entrando com `git add -f`.
- ✓ **Três estratégias:** links com script (ou `stow`), repositório *bare* no home, ou `chezmoi` quando há várias máquinas e segredos.
- ✓ **Segredo não entra no repositório, nem no privado** — `~/.zshrc.local` carregado se existir resolve a maioria dos casos.
- ✓ `apt-mark showmanual` **captura o que os dotfiles não capturam:** os pacotes que você instalou por decisão própria.
- ✓ **Nunca aplicado numa máquina limpa = hipótese.** Teste num contêiner descartável ou num usuário novo — é sempre na primeira vez que aparece o caminho com o seu nome dentro.

NO KALI

A VM do Kali é descartável por natureza, e isso é uma vantagem — se os dotfiles estiverem fora dela. Trate a VM como algo que se joga fora e recria: `git clone` dos dotfiles, bootstrap, e em quinze minutos o ambiente volta. Quem não tem isso acumula anos numa VM que *não pode morrer* — o cenário da abertura. O snapshot cobre o **estado**; os dotfiles cobrem a **intenção**, que é o que se leva para a máquina seguinte. **O home do Kali contém material que nunca pode ir para um repositório:** `~/.msf4/` guarda o banco do Metasploit, com hosts e credenciais de engajamentos; `~/.ssh/` tem chaves; `~/.zsh_history` tem comandos com senhas e endereços de alvos. Um `git init` no home do Kali é, potencialmente, **publicar dados de cliente** — risco técnico e problema contratual, e o argumento mais forte para a lista de inclusão: aqui, o custo de um esquecimento é maior que numa máquina comum. **Os metapacotes substituem metade do bootstrap:** em vez de duzentas ferramentas, `kali-linux-headless` ou `kali-tools-top10` reconstroem conjuntos coerentes com um comando — e o `apt-mark showmanual` lista o *metapacote*, não o que há dentro dele, que é o desejado. **Versione também os scripts de** `~/bin` — as ferramentas pequenas que você escreve são o que mais se perde e mais custa reescrever. **E pense antes de publicar dotfiles em repositório público:** um `.zshrc` revela suas ferramentas, os nomes das suas máquinas e caminhos internos — reconhecimento gratuito para quem se interessar. **Fora do Debian:** o equivalente do `apt-mark showmanual` é `dnf history userinstalled`, e o `chezmoi` expõe `.chezmoi.osRelease.id` como variável de modelo — ramificação declarativa em vez de condicional espalhada.